

Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas - ICEB
Departamento de Computação - DECOM
Ciência da Computação

Simulação de Escopo Dinâmico

BCC202 - Estrutura de Dados 1

Pedro Henrique Menezes Féo de Castro e Daniel Matos Falcão
Professora: Karla Alexsandra de Souza Joriatti

Ouro Preto
18 de fevereiro de 2026

Sumário

1	Introdução	1
1.1	Especificações do problema	1
1.2	Considerações iniciais	1
1.3	Ferramentas utilizadas	1
1.4	Instruções de compilação e execução	1
2	Desenvolvimento	2
2.1	Tipo Abstrato de Dados (TAD)	2
2.2	Algoritmo de Busca e Inserção	2
2.3	Funcionamento do Interpretador	2
2.4	Incluindo fragmento de códigos	2
3	Resultados	5
3.1	Complexidade Temporal	5
3.2	Complexidade Espacial	5
3.3	Discussão dos Resultados	5
4	Considerações Finais	6

Lista de Códigos Fonte

1	Interface do TAD (filaprocessos.h).	2
2	Lógica de Busca Dinâmica (filaprocessos.c).	3
3	Função executar() (filaprocessos.c).	3

1 Introdução

Este trabalho consiste na simulação de um interpretador de código simples com suporte a escopo dinâmico. O objetivo é gerenciar a visibilidade e o tempo de vida de variáveis através de uma pilha de execuções, onde a vinculação de nomes e valores depende da ordem de chamada dos escopos.

1.1 Especificações do problema

O interpretador deve processar comandos de um arquivo de texto seguindo as seguintes regras:

- **begin**: Inicia um novo escopo, empilhando uma nova estrutura.
- **var nome = valor**: Declara uma variável no escopo atual.
- **print nome**: Exibe o valor da variável. Caso não esteja no escopo atual, deve-se buscar nos escopos anteriores da pilha.
- **end**: Finaliza o escopo atual, removendo-o da pilha e liberando sua memória.

1.2 Considerações iniciais

As seguintes ferramentas foram utilizadas no desenvolvimento:

- Ambiente de desenvolvimento do código fonte: Visual Studio Code + Extensão Live Share.
- Linguagem utilizada: C.
- Ambiente de desenvolvimento da documentação: Overleaf L^AT_EX.

1.3 Ferramentas utilizadas

Para validação da implementação e garantia de qualidade:

- *Valgrind*: Utilizado para certificar que toda a memória alocada para as árvores e para a pilha foi corretamente liberada, garantindo a ausência de *memory leaks*.

1.4 Instruções de compilação e execução

Para a compilação do projeto, deve-se utilizar as diretivas especificadas[cite: 473]:

Compilando o projeto

```
gcc -c filaprocessos.c -Wall
```

```
gcc -c tp.c -Wall
```

```
gcc filaprocessos.o tp.o -o exe
```

Opções de compilação:

- *Wall*: Ativa todos os avisos de possíveis erros no código.

Diferente do trabalho anterior, a execução recebe o nome do arquivo diretamente como argumento:

```
./exe nome_do_arquivo_de_teste
```

2 Desenvolvimento

2.1 Tipo Abstrato de Dados (TAD)

A solução foi estruturada utilizando dois conceitos fundamentais de Estruturas de Dados:

- **Pilha de Execução:** Implementada como uma lista encadeada (*LIFO*), onde cada elemento representa um escopo ativo no programa.
- **Árvore Binária de Busca (ABB):** Cada escopo possui uma raiz para uma ABB, que armazena os registros das variáveis (nome e valor) declaradas naquele nível.

As funções principais incluem **criarPilha**, **adicionaPilha** (para o comando *begin*) e **removePilha** (para o comando *end*). A estrutura **Registro** armazena os nomes e valores das variáveis como strings .

2.2 Algoritmo de Busca e Inserção

1. **Inserção (adicionaVar):** Quando uma variável é declarada em um escopo (*var*), ela é inserida na ABB correspondente ao topo da pilha. Se a variável já existir no escopo atual, seu valor é atualizado .
2. **Busca Dinâmica (imprimir):** Ao solicitar o *print*, o interpretador busca a variável na ABB do escopo atual. Caso não a encontre, ele percorre recursivamente os escopos abaixo na pilha até encontrar a definição mais recente ou atingir o escopo global .

2.3 Funcionamento do Interpretador

A função **executar** lê o arquivo palavra por palavra. Ao encontrar *begin*, um novo TAD **Escopo** é alocado e empilhado . Ao encontrar *end*, a árvore do escopo no topo é destruída recursivamente e o escopo é desempilhado .

2.4 Incluindo fragmento de códigos

O Código 1 apresenta a interface do **TAD Pilha de Execução**.

```
1 #ifndef FILAPROCESSOS_H
2 #define FILAPROCESSOS_H
3
4 #include <stdbool.h>
5
6 #define TAM 32
7
8 typedef struct pilhaExecucao PilhaExecucao;
9 typedef struct escopo Escopo;
10 typedef struct no No;
11 typedef struct registro Registro;
12
13 PilhaExecucao* criarPilha();
14 int destroiPilha(PilhaExecucao **ppPilha);
15 void adicionaPilha(PilhaExecucao *pPilha);
16 int executar(const char *nomeArquivo, PilhaExecucao* pPilha);
17 void removePilha(PilhaExecucao *pPilha);
18 void leValor(FILE *file, PilhaExecucao *pPilha);
19 void adicionarNaArvore(PilhaExecucao *pPilha, No *novo);
20 void adicionaRecursivo(No *raiz, No *novo);
21 int procuraArvore(No *raiz, const char nome[TAM]);
22 int imprimir(FILE *file, PilhaExecucao *pPilha);
23 int lePalavras(FILE* file);
24
25 Escopo* criarEscopo();
26 void destroiEscopo(Escopo **ppEscopo);
```

```
27
28 #endif
```

Código 1: Interface do TAD (**filaprocessos.h**).

O Código 3 detalha a lógica de busca dinâmica que caracteriza o interpretador.

```
1 int imprimir(FILE *file, PilhaExecucao *pPilha){
2     char nome[TAM];
3     fscanf(file, "%s", nome);
4
5     Escopo *escop = pPilha->topo;
6     // Percorre a pilha do topo para a base (Escopo Dinamico)
7     while(escop != NULL){
8         // Busca na ABB do escopo atual
9         if(procuraArvore(escop->raiz, nome))
10             return 1;
11         escop = escop->next;
12     }
13
14     printf("Variavel %s nao declarada\n", nome);
15     return 0;
16 }
```

Código 2: Lógica de Busca Dinâmica (**filaprocessos.c**).

```
1 int executar(const char *nomeArquivo, PilhaExecucao* pPilha) {
2     FILE *file = fopen(nomeArquivo, "r");
3     if (!file) {
4         printf("Nao foi possivel abrir o arquivo.\n");
5         return 0;
6     }
7
8     int code;
9     //le o arquivo inteiro
10    while ((code = lePalavras(file)) != EOF){
11
12        switch (code) {
13            // begin
14            case 1:
15                if(pPilha)
16                    adicionaPilha(pPilha);
17                break;
18
19            // end
20            case 2:
21                removePilha(pPilha);
22                break;
23
24            // var
25            case 3:
26                leValor(file, pPilha);
27                break;
28
29            // print
30            case 4:
31                if(!imprimir(file, pPilha))
32                    return 1;
33                break;
34        }
35
36    };
37    return 0;
```

```
38     fclose(file);  
39 }
```

Código 3: Função executar() (**filaprocessos.c**).

3 Resultados

3.1 Complexidade Temporal

A complexidade temporal depende da profundidade da pilha e do balanceamento das árvores de cada escopo.

- **Inserção de Variável:** No caso médio, inserir na ABB custa $O(\log N)$, onde N é o número de variáveis no escopo. No pior caso (árvore degenerada), custa $O(N)$.
- **Busca de Variável (Print):** No escopo dinâmico, o custo é a soma das buscas nas ABBs da pilha. Sendo M o número de escopos, a complexidade é $O(M \cdot \log N)$ em média e $O(M \cdot N)$ no pior caso.
- **Begin/End:** Inserir na pilha é $O(1)$. Destruir o escopo custa $O(N)$, pois percorre todos os nós da árvore para liberar a memória.

A complexidade temporal é, no caso médio:

$$O(M \cdot \log N)$$

3.2 Complexidade Espacial

A complexidade espacial é determinada pelo armazenamento da pilha e de todas as variáveis ativas em todos os escopos.

- Cada variável em um escopo ocupa um nó na ABB: $O(N)$.
- A estrutura da pilha e os ponteiros de escopo: $O(M)$.

A complexidade espacial total é:

$$O(M + N)$$

Este resultado é eficiente, pois o uso de memória é estritamente proporcional à quantidade de variáveis e escopos definidos no programa de entrada.

3.3 Discussão dos Resultados

Os testes confirmaram que a implementação respeita a hierarquia do escopo dinâmico: quando uma variável é redeclarada em um escopo interno, o comando *print* acessa o valor mais recente. Ao fechar o escopo com *end*, o valor anterior volta a ser acessível. A análise com *Valgrind* confirmou que a destruição recursiva das árvores limpa todos os nós corretamente. O sistema de detecção de erros (variável não declarada ou escopo não fechado) funcionou conforme o esperado.

4 Considerações Finais

O desenvolvimento deste trabalho consolidou o conhecimento sobre a aplicação conjunta de **Pilhas** e **Árvores Binárias de Busca**.

Processo de Implementação e Dificuldades: O maior desafio foi coordenar a destruição de memória. Como o escopo dinâmico permite múltiplos níveis, esquecer de liberar uma árvore ao desempilhar um escopo causaria grandes vazamentos. A lógica recursiva para navegação na ABB e destruição de nós foi fundamental para manter o código limpo e modularizado .

Conclusão: O interpretador simulou com sucesso o comportamento de linguagens com escopo dinâmico. O aprendizado principal foi observar como estruturas de dados fundamentais (Pilhas e Árvores) podem ser combinadas para criar ferramentas complexas, como um interpretador de linguagem.

Referências

1. PEDRO HENRIQUE L. SILVA **Aulas sobre: Alocação Dinâmica, TAD, Recursão, Análise de Complexidade de Algoritmos, Árvores Binárias e Pilhas**, BCC202 Estruturas de Dados I. Universidade Federal de Ouro Preto - UFOP.